

AI-POWERED E-COMMERCE PLATFORM WITH INTELLIGENT BUSINESS INSIGHTS

WEEKLY PROJECT UPDATE REPORT 2 - CLASS 5

Abstract - This update established the first complete connection between the React storefront, the Django backend and a locally running AI customer assistant. The team organized the principal shopping pages and backend services, connected the chatbot interface to a Django endpoint, and verified that customer messages could travel from the browser to Ollama and return as visible replies.

1. FRONTEND DEVELOPMENT

Shakib continued the React frontend using Vite and prepared the principal pages required for an e-commerce workflow, including product browsing, product details, categories, cart, checkout, order-related pages, login, registration and AI chat. React Router connected these pages without full browser reloads, while the reusable navigation bar maintained consistent movement across the application.

The customer chat page accepted a typed message, sent JSON to the backend at `/api/ai/chat/`, received the generated reply and displayed it in the interface. This provided the browser-side portion of the first end-to-end chatbot demonstration.

2. BACKEND DEVELOPMENT

Rahat organized the Django backend around separate applications for accounts, store, cart, orders, chat and core services. Database structures were prepared for users, profiles, products, categories, carts, orders and chat-related information. The modular design allowed each area to be developed without placing all logic in one file.

Django REST API work supported communication with the React frontend. Authentication, product management, cart handling and order processing were identified as the main backend responsibilities for the next integration stages.

3. INITIAL AI CHATBOT INTEGRATION

Farhan connected the React chat interface to Django and added a customer chatbot endpoint at `/api/ai/chat/`. The backend accepted a POST request containing the customer message, passed the prompt to the locally installed phi3:mini model through Ollama and returned the generated answer as JSON.

The communication flow was tested as React -> Django -> Ollama -> Django -> React. Successful responses appeared inside the browser chat page, confirming that the frontend, backend and local language model could communicate correctly.

4. TESTING COMPLETED

Testing included starting the Django development server, starting the Vite frontend, checking that Ollama and phi3:mini were available, sending customer prompts and confirming that replies were rendered on the page. Direct API testing was also used to verify that the backend accepted valid JSON messages and returned the expected reply field.

The first integration proved the technical path required for later product-aware and administrator-focused chatbot features.

5. CURRENT PROJECT STATUS

At the end of Update 2, the group had a functional React interface, a modular Django backend foundation and a working customer chatbot connected to a local AI model. The chatbot could hold general conversations, but it did not yet read verified product information from the store database.

Frontend and backend features were still being developed on separate branches, so broader cart, order, account and product integration remained incomplete.

6. NEXT DEVELOPMENT TASKS

The next phase was planned to connect the chatbot with real product records, strengthen the REST API structure, expand authentication and permissions, test the shopping workflow and prepare reliable demonstration data. Further work also included better error handling, documentation and systematic endpoint testing.

7. FILES AND COMPONENTS INVOLVED

Frontend: React chat page, application routes and navigation components. Backend: Django chatbot view and URL configuration, project settings and the Ollama Python dependency. Existing store, account, cart and order applications were kept as the foundation for later integration.

REFERENCES AND TOOLS

[1] React and Vite documentation. [2] Django and Django REST Framework documentation. [3] Ollama Python library documentation. [4] Project GitHub commit history. [5] OpenAI ChatGPT used for explanation, debugging and report preparation; implementation was tested locally by the team.